Immutable Audit — Architecture, Flow and Assurance

SecureWealth Twin · Punjab & Sind Bank Hackathon Series 2026

This document describes the audit subsystem end to end: what is recorded, how it is hashed, signed and replicated, how tampering is detected and repaired, how all of that is surfaced in the admin dashboard, and the ten adversarial scenarios that prove it. Every mechanism described here is implemented and exercised by an automated suite — nothing in this document is aspirational.

> **The claim this system makes.** An audit trail is only worth something if it holds up against someone *inside* the bank. Ours is designed so that a privileged insider with direct database access cannot alter a record undetected — not even by rewriting a majority of the copies.

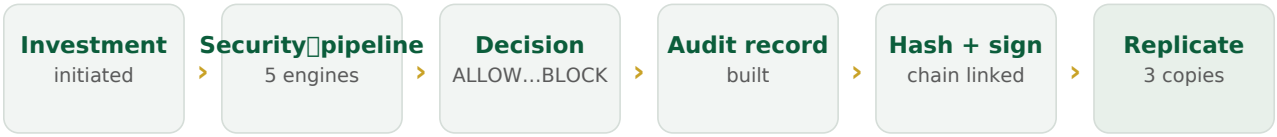
1. Threat model — what we are actually defending against

Most audit designs defend against accidental loss: a disk fails, a row is dropped, so you keep copies. That is not the interesting threat. The interesting threat is a **deliberate, privileged edit** — someone who can reach PostgreSQL directly, bypassing every API check, and rewrite history to hide a decision that was actually made.

Adversary	Capability assumed	Our answer
External attacker	Can call the API, cannot reach the database	Records are written by the pipeline only; there is no API that edits or deletes an audit row.
Privileged insider	Direct `UPDATE` on any table	Hash chain breaks, and the Ed25519 signature fails — they cannot re-sign without the private key.
Insider who knows the design	Edits the master *and* a majority of replicas to agree	Signature authority overrides head-count. One unaltered copy is provably genuine and wins alone.
Key holder	Holds the signing key	**Out of scope.** Stated plainly in section 10 — this reduces to key custody, an HSM problem.

2. What gets recorded, and when

An audit record is written by the security pipeline for every investment decision — not only for refusals. An ALLOW is as much a decision as a BLOCK, and a trail that only records refusals cannot prove what was permitted.



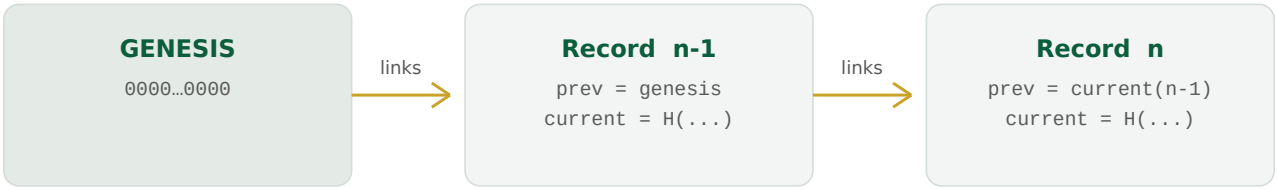
Fields covered by the hash and the signature

Group	Fields
Identity	`audit_id, transaction_id, user_id, device_id, account_id`
Decision	`event_type, final_decision, risk_score`
Money	`amount, channel`
AI verdict	`ai_status, ai_confidence, unknown_risk_severity, unknown_risk_applied, unknown_risk_decision, unknown_risk_reason_code`
Step-up	`challenge_required, challenge_type, challenge_trigger_signal`
Evidence	`security_signals, audit_note, compliance_reference`

Because the AI verdict and the signals that fired are inside the signed payload, an insider cannot later claim the model said something different. **The reasoning is as immutable as the outcome.**

3. Layer one — the hash chain

Each record stores the hash of the record before it, so the log is ordered and self-referencing. The first record links to a fixed genesis value of sixty-four zeroes.



Editing any record changes its current_hash, which breaks every link after it.

```
current_hash = SHA-256( canonical(record)  ||  previous_hash  ||  created_at )
```

Canonical matters: the record is serialised with sorted keys and fixed formatting before hashing, so the same content always produces the same digest regardless of column order or JSON whitespace. Without canonicalisation, verification would produce false alarms on data that had not actually changed.

What this layer gives you: **tamper evidence and ordering**. Change one field in record 40 and every record from 40 onward fails verification. What it does *not* give you: protection against someone who recomputes the whole chain after editing. SHA-256 is public — anyone can recompute it. That is what layer two is for.

4. Layer two — Ed25519 signatures

Every record is signed with an Ed25519 private key. Verification uses only the public key, so the monitoring path never needs the secret.

Property	Value	Why it was chosen
Algorithm	Ed25519 (EdDSA, Curve25519)	Modern, widely vetted, deterministic — no dependence on a random nonce per signature, which is a classic ECDSA failure mode.
Public key	32 bytes	Against 256+ bytes for comparable RSA. Cheap to distribute to every verifier.
Signature	64 bytes, fixed	Fixed size keeps the audit row predictable at high volume.
Speed	Fast sign and verify	Verification runs across the entire chain on a schedule, so per-record cost compounds.

This is the layer that stops a knowledgeable insider. Recomputing SHA-256 is free. Producing a valid Ed25519 signature is not — it requires the private key. An attacker can make the chain *look* consistent; they cannot make it *verify*.

5. Layer three — four independent copies

Every record is written to the master table and to three replica tables (`audit_replica_1/2/3`), each carrying the same content, hash and signature. Replication exists so that a damaged copy can be *repaired*, not merely detected.

State	Meaning	Action
HEALTHY	Matches the established truth	None
CORRUPTED	Differs from it; mismatched fields are listed	Overwritten from a healthy copy
REPLICA_BEHIND	Record has not arrived yet	Explicitly **not** treated as corruption — lag is normal and must not raise a false alarm

6. Verification — and the rule that makes this different

When copies disagree, something has to decide which one is the truth. This is the design decision that matters most, and the obvious answer is wrong.

Approach	How it decides	Fails when…
Trust the master	Replicas are compared against `audit_logs`	**The realistic attack.** An insider edits the master; all three honest replicas are declared corrupt and 'repaired' with the forgery. The tamper is propagated, not healed.
Content majority	All four copies vote; the most common content wins	An attacker who edits 3 of 4 copies to the same value holds the majority and wins outright.
Signature authority what we implement	Only copies whose Ed25519 signature verifies are eligible to be the truth	Only if the private key is stolen. Forged copies fail verification no matter how many there are.

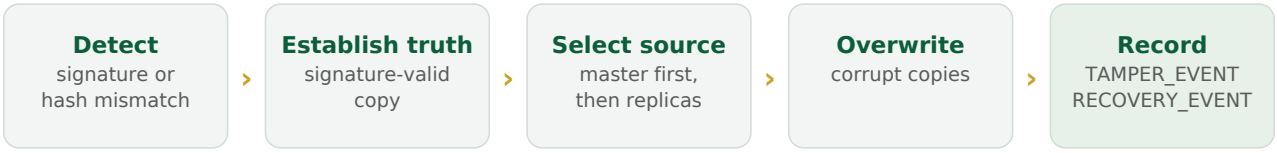
The consequence, stated precisely

Under signature authority a **single** signature-valid copy is sufficient to restore from, because it is provably genuine. Under majority you would need at least two agreeing copies. The system therefore sets its recovery threshold differently depending on which authority was used, and reports which one applied.

Authority in force	Copies needed to recover	Reported as
Cryptographic signature	**1** valid copy	`cryptographic_signature`
Content majority (degraded)	**2** agreeing copies	`content_majority`

The degraded path exists deliberately: if no public key is configured, the system still self-heals rather than refusing to operate. It reports the weaker authority honestly instead of implying a guarantee it is not providing.

7. Recovery — and why recovery is itself audited



Repair is not silent. Both the detection and the repair are written back into the audit log as `TAMPER_EVENT` and `RECOVERY_EVENT` records, which are themselves hashed, signed and replicated.

> **Why this detail matters to an auditor.** A system that quietly fixes itself destroys the evidence that anything happened. Recording the tamper and the repair means the incident survives the fix — you can answer 'was this record ever altered?' months later, not just 'is it correct now?'

8. Continuous monitoring

Verification does not wait for someone to open the dashboard. A background scheduler runs the tamper monitor every **300 seconds**, walking the full chain.

The sweep checks	And reports
Chain linkage across every record	Chain status, records checked, and the first `audit_id` where it breaks
Ed25519 signature on every record	Counts of valid and invalid signatures, with an `INVALID_SIGNATURE` finding per record
All three replicas of every record	Replica health counts and the specific mismatched fields
Recovery outcomes	Recoveries attempted versus recoveries succeeded — deliberately separate numbers

Attempted and succeeded are tracked separately on purpose. A repair that fails is a materially different situation from one that was never needed, and collapsing them into a single 'healthy' count would hide it.

9. Integration — API surface and admin dashboard

9.1 Endpoints

Endpoint	Returns	Consumer
`GET /audit/chain`	The ordered chain of records	Audit Log tab
`GET /audit/integrity`	Live chain + signature verification result	Integrity seal
`GET /audit/replicas/{id}`	Per-copy status, mismatched fields, authority used	Record drill-down
`GET /audit/recovery-history`	Past tamper and recovery events	Audit Defence feed
`GET /audit/summary`	Aggregate counts for the header cards	Overview tab
`GET /admin/audit`	Search with decision, risk and pagination filters	Audit Console
`GET /admin/redteam/scenarios`	The scenario catalogue	Red Team tab
`POST /admin/redteam/run/{id}`	Executes one scenario, returns a step timeline	Red Team tab

All administrative endpoints require an admin key; a wrong or missing key returns **401**, and both cases are covered by the automated suite.

9.2 What each dashboard tab shows

Tab	Contents
Overview	Total audit records, average risk score, records in the last 24 hours, blocked amount, total invested, and the decision split across ALLOW / WARN / CHALLENGE / BLOCK. Live fraud alerts poll continuously.
Audit Console	Paginated, filterable audit log — by decision and by minimum risk score. Each row drills down to per-replica status. Carries the **chain integrity seal**, which turns red the moment verification fails.
Investments	Investment history with status filtering, reconciled against the audit trail so the money view and the decision view cannot silently diverge.
Red Team	The ten scenarios below. Each runs a *real* attack against the running system and renders a step-by-step timeline, followed by an **Audit Defence** feed of the resulting tamper and recovery events.

10. The red team suite — ten live scenarios

These are not mocked results. Audit scenarios write directly to PostgreSQL, bypassing the API exactly as an insider would. Fraud scenarios go through the full transaction pipeline. Phishing scenarios hit the live engine. Runs are serialised behind a lock, because two scenarios mutating balances and the chain at once would produce misleading output.

#	Scenario	What it actually does	Expected outcome	Steps
1	**Insider Audit Tampering** Audit Integrity · critical	Edits the `audit_logs` master row directly in PostgreSQL to erase a fraud decision, bypassing the API entirely.	Signature verification detects the altered master and restores it from a valid copy.	5
2	**Replica Corruption** Audit Integrity · high	Silently corrupts one replica at the database level.	The corrupt replica fails its signature check and is repaired.	5
3	**Mass Forgery — 3 of 4 Copies** Audit Integrity · critical	Edits the master **and** two replicas to the same forged value, holding a 3-of-4 majority.	Signatures override head-count: the single unaltered copy is provably genuine and the forgery is reversed.	6
4	**Tamper & Leave Broken** Audit Integrity · critical	Tampers the chain and deliberately *skips* healing, so the compromised state stays visible.	The integrity panel turns red and stays red until a repair sweep is run.	4
5	**Account Takeover — Large Transfer** Fraud Detection · high	Attempts an investment roughly 25× the customer's normal size using a valid session.	Decision escalates to CHALLENGE or BLOCK; no money moves.	5
6	**Rapid Drain Attempt** Fraud Detection · high	Fires five investments back to back to drain the account quickly.	Velocity signals fire and later attempts are escalated.	8
7	**Zero-Day / Unknown Pattern** Fraud Detection · medium	Submits a transaction unlike anything in the customer's history — unusual hour, unseen device, extreme amount.	The novelty detector classifies it ANOMALOUS or UNKNOWN_RISK.	4
8	**Phishing Campaign** Social Engineering · medium	Pushes five real-world scam SMS patterns plus one genuine message through the engine.	Every scam scores HIGH or MEDIUM; the genuine message stays SAFE.	6

#	Scenario	What it actually does	Expected outcome	Steps
9	**Credential Brute Force** Access Control · high	Repeated login attempts with wrong MPINs against a valid customer ID.	Every attempt is rejected and failures are counted.	3
10	**Full Integrity Sweep** Audit Integrity · info	Re-hashes and re-verifies every audit record and all replicas.	Chain reported HEALTHY with all signatures valid. Also repairs damage left by scenario 4.	2

Scenario 4 is intentionally destructive — and order matters. It leaves the chain broken on purpose, to prove the integrity panel is a live signal rather than a static badge. Run **Full Integrity Sweep** afterwards to repair before demonstrating anything else, or later scenarios will report pre-existing damage as their own.

11. Verification status

The audit subsystem is covered by automated suites that run against the live backend. Most recent full run:

Suite	Covers	Result
`dashboard_test.sh`	All admin endpoints, authentication rejection, every red-team scenario, post-attack integrity	**35 / 35** 10 of 10 scenarios DEFENDED
`smoke_test.sh`	Audit record written per investment, chain and signature verification, customer activity feed	**60 / 60**
`audit_phase_test`	Hash chain, signatures, replica consensus, recovery paths	**36 / 36**
`frontend_test.sh`	Response shapes the customer-facing activity screen depends on	**37 / 37**

After the full attack sequence the chain reported **21 records, 0 invalid signatures**, with **8** tamper and recovery events available in the Audit Defence feed — the incidents survived their own repair, as intended.

12. Honest limitations

Stating these is not a weakness in the design; concealing them would be.

Limitation	Assessment
Key custody is the real boundary	Signature authority reduces the whole guarantee to ‘the private key is safe’. In production that key belongs in an HSM or KMS with its own access trail. Today it is loaded by the service, which is appropriate for a prototype and not for a bank.
All four copies share one database	The replicas are separate tables, not separate hosts. That defends against targeted row edits, which is the modelled threat, but not against dropping the whole database. Physical separation is a deployment change, not a redesign — the verification logic is already replica-agnostic.
Deletion versus alteration	The chain makes a deleted record obvious, because the link from its successor no longer resolves. Restoring it depends on at least one replica retaining the row.
No external anchoring	Chain heads are not published to an outside system. Anchoring periodic checkpoints externally would close the theoretical case where an attacker with the key rebuilds history wholesale.

Prepared for the SecureWealth Twin team. Every mechanism, threshold and test figure in this document was read from the implementation and from live test output — none of it is estimated.